



Advanced Data Analysis with R

From Prompt to Package

Sebastian Hanß

The Plan

-  Let's talk LLMs: Tutor, Tool, or Trap?
-  Build Your Own R Package
-  Adding a Function and Documentation
-  Testing Your Package with **testthat**
-  Package Checking, Installation, and Usage
-  Publishing Your Package on GitHub
-  Automations with GitHub Actions
-  Wrap-up and Q&A

Let's talk LLMs: Tutor, Tool, or Trap?



LLMs as Tools for Study, Coding, and Research

-  Non-judgemental tutor
-  Thinking partner for understanding and structuring ideas
-  Only helpful if you use them engaged and critically, not as a push-button generator

How LLMs Learn: Training

- Collect a massive text corpus → tokens

On the Origin of Species by Means of Natural Selection, or the Preservation of Favoured Races in the Struggle for Life. When on board H.M.S. 'Beagle,' as naturalist, I was much struck with certain facts in the distribution of the inhabitants of South America...

On the Origin of Species by Means of Natural Selection, or the
Preservation of Favoured Races in the Struggle for Life. When on
board H.M.S. 'Beagle', as naturalist,

Learning Patterns at Different Scales

Token-level patterns:

Token	Count
the	32
species	7
origin	3

Combination-level patterns:

Token sequence	Count
of species	4
the origin	2
struggle for life	1

How LLMs Learn: Training

- Collect a massive text corpus → tokens
- A.I learns correlations and patterns between **tokens**
- Learns patterns in text and structure of language
 - Given tokens → likely next token
- Result: a huge **pattern machine**

No database, no token lookup table

- LLM itself represents language patterns
- LLM has no data or knowledge or semantics

How LLMs Answer: Prediction

- Input: your prompt → tokens
- Model predictions are based on the **context**: your text and its replies
- Predicts one token at a time (!) . . .

Darwin example: “...*the inhabitants of South*”:

Next token	~Probability
America	85%
Asia	7%
African	4%

Next token	~Probability
------------	--------------

American	3%
----------	----

Context matters!

Example: What is the Blauer Turm in Göttingen?

Mental Model: Pattern Machine, Not Oracle

- Learned **patterns**, not meaning
- Predicts **tokens**, doesn't understand¹ text
- Not a database of truth (no matter how it sounds)

1. You could argue whether pattern-based text prediction counts as a kind of 'understanding'. For our purposes, the safe mindset is: it doesn't have human-like understanding, so we must **verify** everything.

LLMs Leverage Your Prior Knowledge

- No prior knowledge → high risk of nonsense
- Some prior knowledge → you can:
 - provide better context
 - judge answers
 - clarify & extend what you know

Always stay in the Driver's Seat

- You define the **goal** and desired **output**
- LLM's role: suggest, explain, critique – not “do it for you”
- Treat answers as **proposals**, not decisions
- Never blindly trust LLMs (like *any* other source)
- If you can't recognize your own thinking in the text, it's not your text, yet

An Effective Workflow for Working with LLMs

A simple loop that keeps you in control:

- Clarify your **goal** and desired **output**
- Give enough **context**
- Work **one concrete task at a time**
- **Check & understand** every answer
- **Summarize & log** → then iterate (ping-pong)

Privacy & Ethics:

Use LLMs Responsibly

- Don't paste **sensitive or identifiable data** into public LLMs
- Follow the **university's rules** on AI use and plagiarism
- You are responsible for any **claims, code, and text** you submit
- Be aware: LLMs reflect **biases** in their training data

Tutor, Tool, or Trap?

- LLMs are **pattern machines** that amplify your prior knowledge
- They become a **trap** when you:
 - accept hallucinations
 - hand in text or code you don't truly understand
 - skip building your own mental model
- They become a **tool / tutor** when you:<
 - stay in the **driver's seat** with clear goals and small steps
 - use them to **deepen**, not replace, your thinking
 - treat LLM drafts as raw material and **revise, cut, and reorganize them until the text is truly yours**

Q&A

Digitales Lernen und Lehren

LLMs for Coding

LLMs change the **cost** of writing code but you are responsible.

What stays your job:

- Defining **what** the code should do
- Knowing **how to check** if it does
- Understanding the **structure** that keeps it maintainable

Software engineering — tests, documentation, CI — is what lets you stay in the driver's seat.

**Break (15 min) 🚀 Grab a coffee,
stand up and stretch!**



Build Your Own R Package



Why Build Your Own R Package?

```
1 final_script.R  
2 final_script_v2.R  
3 final_script_v2_final.R  
4 landscape_calc_v3.R
```

- Where did this code come from?
- What does this function do again?
- Which R-packages and versions did I use?
- Why did it work 3 months ago but not now?

Organize Your Code as a Package

```
1 install.packages("forestMetrics") # or from GitHub
2 library(forestMetrics)
3
4 load(file='landscape.rda')
5
6 landscape_class_area <- calc_area(landscape)
```

- ✓ Clear structure
- ✓ Reusable functions
- ✓ Auto-generated help pages
- ✓ Built-in documentation and tests

You don't need to publish on CRAN to benefit.

 **Create a Research Tool, not a
Collections of Scripts**

Tools We Use

- **devtools**: build, install, test
- **usethis**: automate setup tasks
- **roxygen2**: documentation
- **testthat**: testing
- **RStudio**: integrated development

Create a New R Package

```
1 # Load devtools and usethis
2 library(devtools)
3 library(usethis)
4
5 create_package("forestMetrics", check_name = FALSE) # adjust path and name
6
7 # RStudio will create a new project and open it for you
8 ## Load devtools and usethis for the new R session
9 library(devtools)
10 library(usethis)
11
12 use_package("terra")           # add dependencies
13 use_git()                     # initialize git repository, needs to resta
```

devtools

devtools can be a bit tricky to install.

- Ubuntu users may need to install `libcurl4-openssl-dev` `libssl-dev` `libfontconfig1-dev` `libharfbuzz-dev` `libfribidi-dev` `libxml2-dev` `libfreetype6-dev` `libpng-dev` `libtiff5-dev` `libjpeg-dev` before installing `devtools` in R.
- Windows users may need to install `Rtools`. If nothing works, you can use rstudio.gwdg.de or <https://jupyter.hpc.gwdg.de> (docs [here](#)).

terra

terra might also be tricky to install, especially on Linux. If you run into issues, have a look at their [installation instructions](#).

Set Up Package Metadata

Open the `DESCRIPTION` file and update:

- Title
- Description
- Authors
- License (optional, use `use_mit_license()`, for example)

```
1 # Example
2 use_mit_license("Your Name")
3 use_package("terra") # see `?use_package` on how to add a specific version
```


Add a Function

```
1 # Create new R script for the function
2 use_r("total_area")
3
4 # Add function
5 total_area <- function(landscape) {
6   stopifnot(inherits(landscape, "SpatRaster"))
7   # calculate patch areas (simplified)
8   area_ha <- terra::cellSize(landscape, mask = TRUE, unit = "ha")
9   total <- sum(terra::values(area_ha), na.rm = TRUE)
10
11   return(total)
12 }
```

Use `total_area`

```
1 # load example data (download from Git)
2 landscape <- terra::rast(matrix(1, 10, 10)) # example raster
3 # We cant use total_area directly
4 total_area(landscape) # does not work (yet)
5 forestMetrics:::total_area(landscape) # access internal functions
```



Documentation and Export

Click on the function name (i.e. `total_area`) to move the cursor there. Then, Ctrl + Alt + Shift + R (or Cmd + Option + Shift + R on Mac) to generate the documentation template.

Alternatively, you can use the menu **Code -> Insert Roxygen Skeleton** or write the documentation manually using `roxygen2` syntax.

```
1 #' Compute total area of a landscape
2 #'
3 #' @param landscape SpatRaster, the landscape raster (terra SpatRaster object)
4 #'
5 #' @return Total area in hectares (ha), omitting NA values.
6 #' @examples
7 #' library(terra)
8 #' landscape <- terra::rast(matrix(1, 10, 10)) # example raster
9 #' total_area(landscape)
10 #' @export
```

Generate documentation

```
1 # Generate documentation
2 devtools::document()
```

This will create a `man/` directory with documentation files for your functions. The `@export` tag makes the function available to users of your package. Have a look at the `man/` directory to see how the documentation is structured. Also your `NAMESPACE` file will be updated to include the new function.

You can also use the menu `Build -> More -> Document` in RStudio or configure your build tools to automatically run `devtools::document()` when you build your package: `Build -> Configure Build Tools...` and check the box for “Document on build”.

Load and Test

```
1 # Load package
2 devtools::load_all() # or use the Install button in RStudio (build tab in t
3
4 # Try out the function
5 ?total_area
6
7 landscape <- terra::rast(matrix(1, 10, 10)) # example raster
8 landscape |> total_area() # works now!
```

Add a vignette

You can add a vignette to your package to provide a longer-form documentation or tutorial. Vignettes are typically used to explain how to use your package or to provide examples of its functionality. You can create a vignette using the `use_vignette()` function from `usethis`:

```
1 use_vignette("forestMetrics") # creates a vignettes/ directory with a templ
```

Add a README file

You can add a README file to your package to provide an overview of your package, its functionality, and how to use it. The README file is typically written in Markdown format and is displayed on the package's GitHub page or CRAN page. You can create a README file using the `use_readme_rmd()` function from `usethis`:

```
1 use_readme_rmd() # creates a README.Rmd file
```

**Break (15 min) 🚀 Grab a coffee,
stand up and stretch!**

Testing Your Package with testthat

Software Testing: Why?

- Catch bugs early <- save time in the long run
- Improves reliability, confidence, and trust
- Testable code is inherently better code

Software Testing: What?

- **Unit tests:** test individual functions
- **Integration tests:** test how program components work together
- **End-to-End tests (E2E):** test the entire workflow

Unit Tests

- Functions to test functions, usually named `test-functionname()`
- Have no parameters
- Fixed data and expected results
- returns a boolean value - `TRUE` or `FALSE`
- Use `testthat` to write unit tests

Integration Tests

- Not our focus today because they are more complex
- `testthat` can also handle these by looping through multiple inputs

E2E Tests

- Not our focus today because they are highly specific
- `testthat` can also be used by defining and testing pipelines

Software Testing: How?

- Use the `testthat` package
- Have at least one test for each function in your `R/` directory
- When you find a bug, write a test that fails first, then fix the bug
- Use `usethis::use_test()` to write tests in `tests/testthat/`
- Run tests with `devtools::test()`

Package Checking, Installation, and Usage

- Check your package: `devtools::check()`
- Install your package: `devtools::install()`
- Load your package: `library(forestMetrics)`

Add a Unit Test

```
1 library(testthat)
2 library(usethis)
3
4 use_testthat() # Initialize testthat framework, creates a tests/testthat/ d
5 use_test("total_area") # Create a test test-total_area.R file for the total
6
7 # Inside test-total_area.R
8 test_that("Total area calculation", {
9   # Load example data
10   landscape <- terra::rast(matrix(1, 10, 10)) # you might want to test a mo
11
12   # Test the function
13   result <- total_area(landscape)
14
15   # Check if the result is a numeric value (this is the first unit test)
16   expect_type(result, "double")
17
18   # Check if the result is greater than zero (assuming landscape has some a
```

This code creates the test infrastructure using `usethis::use_testthat()`. This will set up the `tests/` folder

and add `testthat` to the `Suggests` in `DESCRIPTION` (since tests are typically not needed by the user of the package, just for development). It also creates a template file `tests/testthat.R` which is needed to run the tests. The actual tests are written in `tests/testthat/test-your_function_name.R`.

Now you can run the tests using:

```
1 devtools::test()
```

Try to deliberately break the test by changing the `expect_true(result > 0)` to `expect_true(result < 0)` and run `devtools::test()` again. You should see that the test fails, which is expected.

Testing is an important part of package development, especially when you plan to collaborate and share your package with others. It helps you catch bugs early and ensures

that your functions behave as expected. You can write multiple tests for different aspects of your function, and you can also use `testthat` to test for specific conditions, such as errors or warnings.

More checks, building and installation

You can check your package for common issues with `devtools::check()`. This will run a series of checks on your package, including checking the documentation, tests, and code style. It will also check if your package can be installed and loaded correctly. If you plan to submit your package to CRAN, `devtools::check()` needs to pass without any errors or warnings.

```
1 devtools::check() # takes a while...
```



Publishing Your Package on GitHub

Why GitHub?

The easiest way to share your package with others is to upload it to GitHub. You can use the `usethis` package to set up a GitHub repository for your package.

```
1 library(usethis)
2 use_git() # Initialize a git repository if not already done
3 use_github() # This will prompt you to create a new GitHub repository
```

Now others can install your package using `devtools::install_github("yourusername/forestMetrics")`.

Test Automation with GitHub Actions

What is GitHub Actions?

- Runs automated tasks **in the cloud** on every push or pull request
- Free for public repositories
- Defined as YAML files in `.github/workflows/`

`R CMD check` runs automatically on Linux, macOS, and Windows — every time you push.

Set Up R-CMD-check

```
1 usethis::use_github_action("check-standard")
```

- Creates `.github/workflows/R-CMD-check.yaml`
- Runs your tests and `devtools::check()` on every push
- Fails visibly if something breaks

Add a Badge to Your README

```
1 usethis::use_github_actions_badge("R-CMD-check")
```

Adds to `README.md`:

```
1 [![R-CMD-check](https://github.com/yourname/forestMetrics/actions/workflows
```

- Shows build status publicly
- Readers see at a glance: *does this package work right now?*



Package Website with pkgdown

What is pkgdown?

- Turns your R package into a **static website**
- Auto-generates from: function docs (roxygen2), vignettes, [README.md](#)
- Hosted for free on **GitHub Pages**

Set Up and Deploy in One Command

```
1 usethis::use_pkgdown_github_pages()
```

This does everything:

1. Creates `_pkgdown.yml` (site configuration)
2. Adds a GitHub Actions workflow — site rebuilds on every push
3. Configures GitHub Pages to serve from the `gh-pages` branch

What You Get

- **Reference page:** all functions with full docs
- **Articles:** your vignettes, rendered as web pages
- **Changelog:** from `NEWS.md`
- **URL:** <https://yourname.github.io/forestMetrics/>

Every push rebuilds and redeploys the site automatically.



Putting It All Together

What You Built Today

-  An R package with reusable, documented functions
-  Documentation — roxygen2 → man pages + pkgdown website
-  Unit tests with **testthat**
-  CI: `R CMD check` + status badge on every push
-  A documentation website, deployed automatically on every push

Why This Matters for LLMs

Recall from this morning: LLMs are **pattern machines** — fluent, but not reliable.

The package structure gives you the tools to **verify** LLM-generated code:

- Tests catch wrong output
- `R CMD check` catches broken APIs and missing exports
- CI makes invisible bugs visible before your collaborators notice



Wrap-up and Q&A

- <https://r-pkgs.org>
- <https://style.tidyverse.org>
- <https://testthat.r-lib.org>
- <https://usethis.r-lib.org>

If you have any questions, please feel free to drop me an e-mail: shanss@uni-goettingen.de